Mondragon Unibertsitatea

Escuela Politécnica Superior

Reinforcement Learning -RL-

Deep Reinforcement Learning DQN

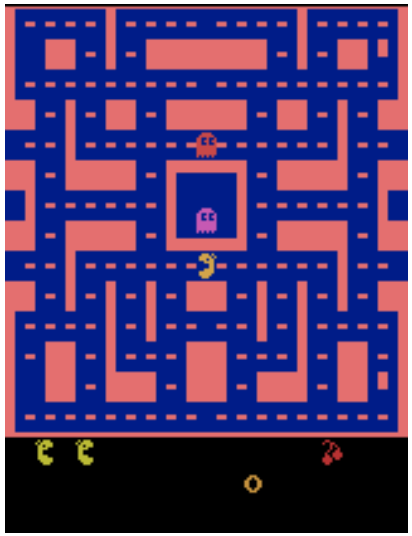
Aitor Duo Zubiaurre
aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)
T. +34 670 254 555
www.mondragon.edu

Deep Reinforcement Learning

Mondragon Unibertsitatea
Escuela Politécnica Superior

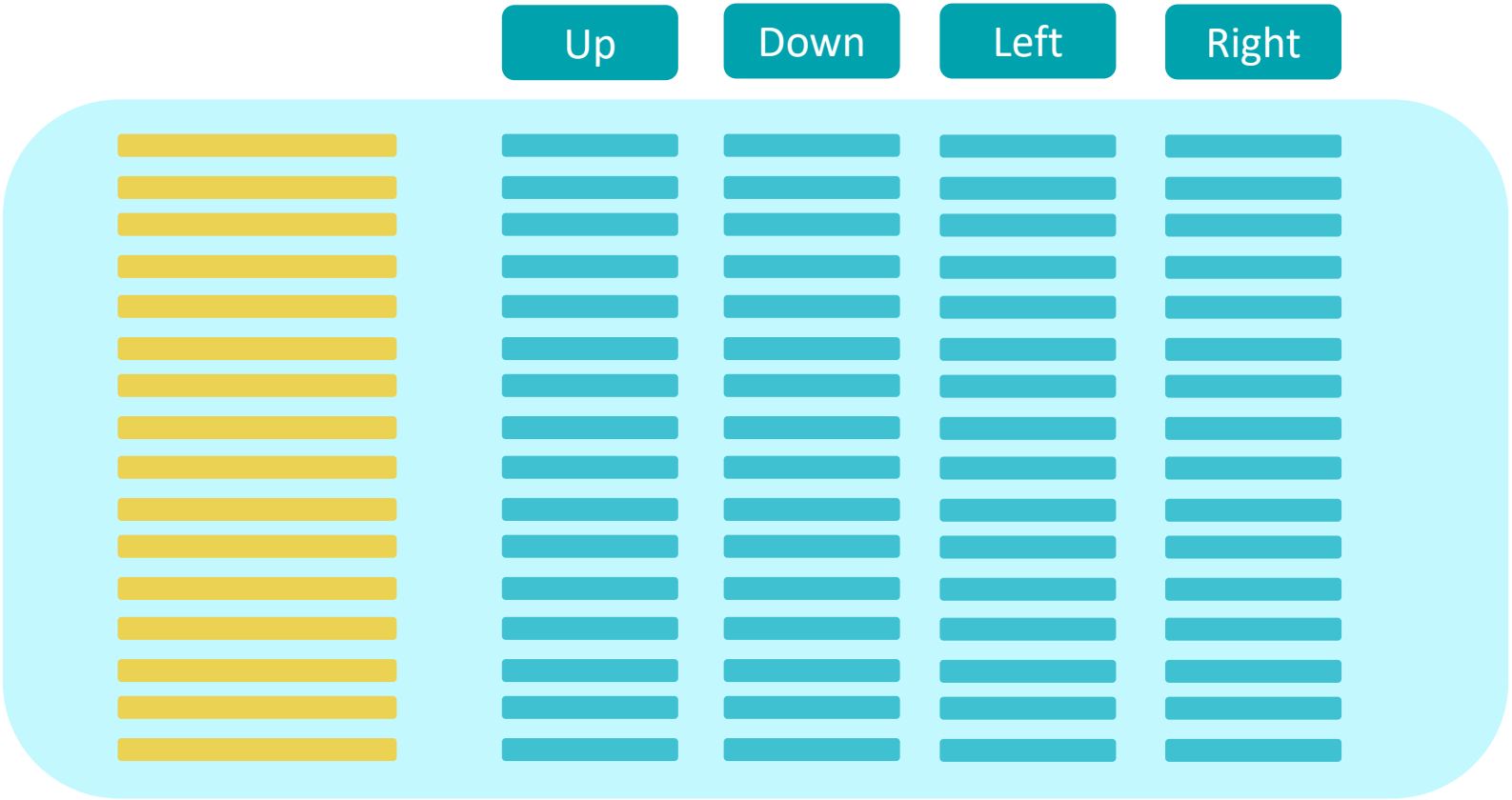
- Traditional RL is suitable for low dimensional tasks.
- Many traditional applications have high-dimensional state and action space.



Deep Q-learning

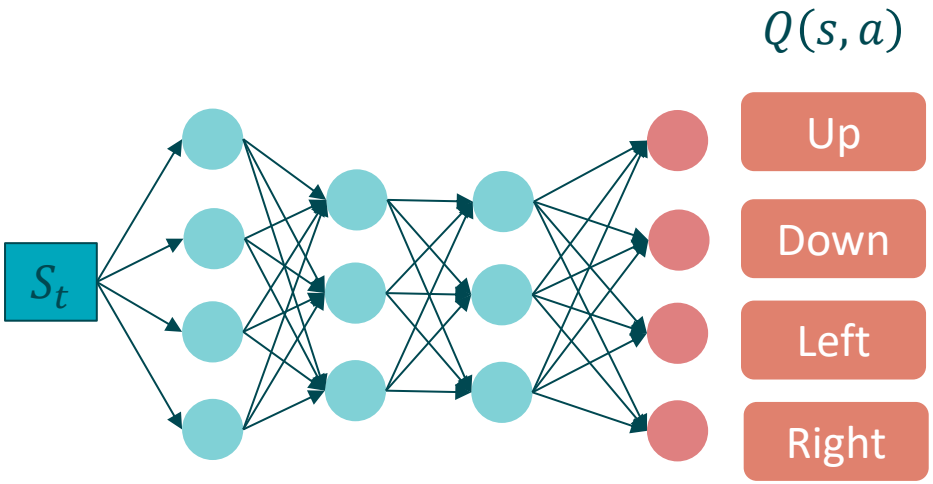
- What is Deep Q-learning?

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma \max Q(s',a') - Q(s,a)]$$



Deep Q-learning

- DQN
  - We'll use a deep neural network to estimate the 'Q' values for each combination of 'state' and 'action' in a specific environment. The network will then estimate the best 'Q' function.
  - The loss from the network is calculated by comparing the outputted Q-values to the target Q-values from the right hand side of the Bellman equation, and as with any network, the objective here is to minimize this loss.



Deep Q-learning

- **Input**
  - On Frozen Lake, we could easily represent the states using a simple coordinate system from the grid of the environment and use this as input.

```
SFFF
FHFH
FFFH
HFFG
```

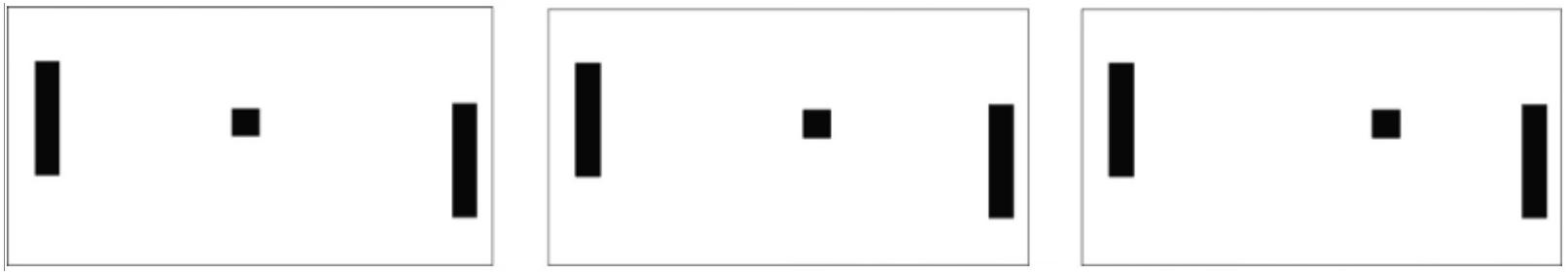
  - If we're in a more complex environment, like a video game, for example, then we'll use images as our input.
  - Specifically, we'll use still frames that capture states from the environment as the input to the network.
  - Rather than having a single frame represent a single input, we usually will use a stack of a few consecutive frames to represent a single input.

Can you say which is the ball direction?

Deep Q-learning

Mondragon Unibertsitatea
Escuela Politécnica Superior

- Input



Now, which way is the ball going?

Deep Q-learning

Mondragon Unibertsitatea
Escuela Politécnica Superior

- **Experience replay and replay memory**
  - We use a technique called experience replay when training with deep Q-networks. This involves storing the agent's experiences at each time step in a data set called the replay memory.
  - At time t the agents experience is expressed as:

$$e_t = \{s_t, a_t, r_{t+1}, s_{t+1}\}$$

  - In practice, we'll usually see the replay memory set to some finite size limit, N, and therefore, it will only store the last N experiences.
  - This replay memory data set is what we'll **randomly sample** from to train the network.

Deep Q-learning

- Why use experience replay?
  - Why would we choose to train the network using random samples from replay memory, rather than just providing the network with the experiences as they occur in the environment?

A key reason for using replay memory is to break the correlation between consecutive samples.

Other reason is to avoid catastrophic forgetting, a neural network tends to forget the previous experiences as it gets new experiences. If the agent is in the first level and then in the second (different), it can forget to behave and play the first level.

Deep Q-learning

- Loss

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma \max Q(s',a') - Q(s,a)]$$

New q-value estimation

Former q-value estimation

Learning rate

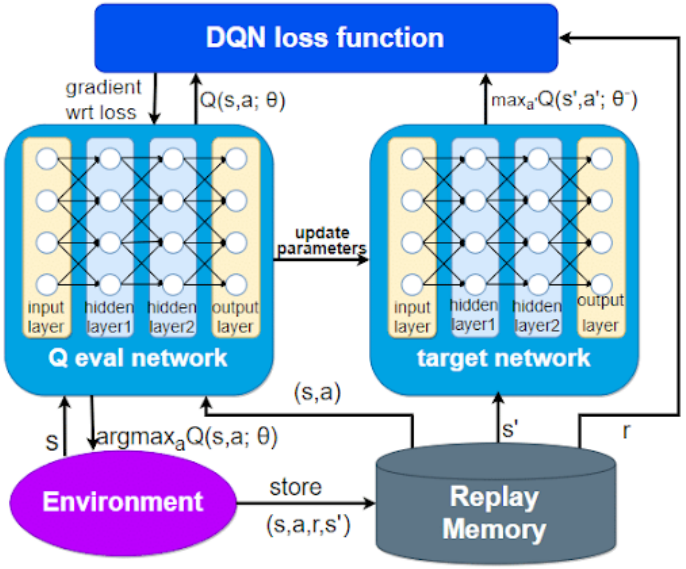
Immediate reward

Discounted estimate optimal Q-value of next state

Former q-value estimation

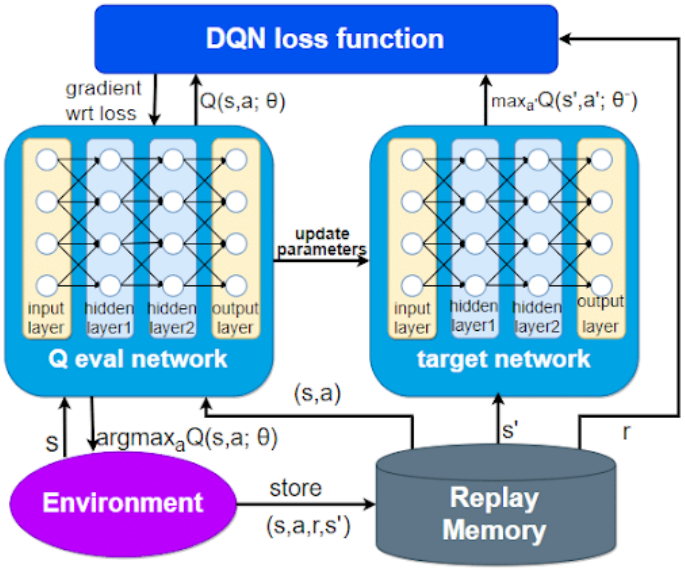
TD target

TD error



Deep Q-learning

- The training loop



Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$
For episode = 1, M **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  For $t = 1$,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

    Sampling

    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q} = Q$

    Training

  End For
End For

Deep Q-learning

Mondragon Unibertsitatea
Escuela Politécnica Superior

- Stable Baselines3
  - https://stable-baselines3.readthedocs.io/en/master/#
  - Is a set of reliable implementations of reinforcement learning algorithms in PyTorch
  - Most of the library tries to follow a sklearn-like syntax for the Reinforcement Learning algorithms.

Deep Q-learning

Mondragon Unibertsitatea
Escuela Politécnica Superior

- Basic Example Code

```
import gymnasium as gym

from stable_baselines3 import DQN

env = gym.make("CartPole-v1", render_mode="rgb_array")

model = DQN("MlpPolicy", env,device="cuda", verbose=1)

model.learn(total_timesteps=300_000)

vec_env = model.get_env()
obs = vec_env.reset()

for i in range(1000):
    action, _state = model.predict(obs, deterministic=True)
    obs, reward, done, info = vec_env.step(action)
    vec_env.render("human")
```

Deep Q-learning

Mondragon Unibertsitatea
Escuela Politécnica Superior

- **Logger**
  - Eval/
    - **mean_ep_length:** Mean episode length
    - **mean_reward:** Mean episodic reward
  - Rollout/
    - **ep_len_mean:** Mean episode length (over last 100 episodes)
    - **ep_rew_mean:** Mean episodic training reward
    - **exploration_rate:** Current value of the exploration rate when using DQN
  - Time/
    - **episodes:** Total number of episodes
    - **fps:** Number of frames per seconds
    - **iterations:** Number of iterations
    - **time_elapsed:** Time in seconds since the beginning of training
  - Train
    - **loss:** Current total loss value
    - **learning_rate:** Current learning rate value

```
----------------------------------
| eval/               |          |
|    mean_ep_length   | 225      |
|    mean_reward      | 225      |
| rollout/            |          |
|    exploration_rate | 0.0726   |
| time/               |          |
|    total_timesteps  | 100000   |
| train/              |          |
|    learning_rate    | 0.00326  |
|    loss             | 0.0293   |
|    n_updates        | 22499    |
----------------------------------
```

https://stable-baselines3.readthedocs.io/en/master/common/logger.html

Deep Q-learning

- **Limitations**
  - DQN is designed for problems with discrete action spaces.
  - The Q-table approximation doesn't work well when there are too many possible actions.
  - DQN is an off-policy method, so using a replay buffer can cause problems with non-stationarity in training.
  - Requires a large replay buffer, increasing memory usage and training complexity.
  - DQN can overestimate Q-values, leading to suboptimal policies.
  - Training can be slow, requiring **epsilon-greedy exploration** for long periods.

Mondragon Unibertsitatea
Escuela Politécnica Superior

Eskerrik asko
Muchas gracias
Thank you

Aitor Duo Zubiaurre
aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)
T. +34 670 254 555
www.mondragon.edu